



**UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO
CAMPUS PETROLINA**

Éricles Barros de Sá

ROBÓTICA I

Método da Janela Dinâmica

Petrolina, PE

2026

Éricles Barros de Sá

ROBÓTICA I

Método da Janela Dinâmica

Trabalho requerido pelo professor Juracy Emanuel França, como parte das exigências de avaliação da disciplina optativa de Robótica 1, do curso de Engenharia da Computação, da Universidade Federal do Vale do São Francisco.

Petrolina, PE

2026

SUMÁRIO

1. Introdução.....	4
2. O Método da Janela Dinâmica (DWA).....	4
3. Metodologia e Implementação.....	6
4. Resultados e Discussão.....	9
5. Anexo: Código Fonte (Python).....	10
6. REFERÊNCIA.....	15

1. Introdução

Este relatório descreve a implementação, os testes e os resultados da aplicação do algoritmo de navegação autônoma conhecido como Método da Janela Dinâmica (*Dynamic Window Approach* - DWA). O objetivo do trabalho consistiu em fazer com que um modelo 3D do robô **iRobot Create 2**, inserido no simulador **CoppeliaSim**, fosse capaz de desviar de obstáculos desconhecidos e alcançar uma coordenada alvo (*Target*) no mapa.

Para contornar limitações de processamento interno e garantir maior escalabilidade, a lógica de controle foi desenvolvida externamente em linguagem **Python** utilizando a biblioteca **ZeroMQ Remote API**, permitindo a comunicação em tempo real entre a IDE (VS Code) e a *engine* de física do simulador em modo síncrono.

2. O Método da Janela Dinâmica (DWA)

O DWA é um algoritmo de planejamento de trajetória local reativo. Ele não calcula a rota completa até o final do mapa, em vez disso, ele atua no horizonte de tempo imediato (geralmente 1 segundo no futuro).

Matematicamente, o espaço de busca efetivo e válido de velocidades, denominado Espaço de Velocidades Resultante V_r , é formalizado através da interseção estrita de três conjuntos independentes de restrições físicas: V_r , é formalizado através da interseção estrita de três conjuntos independentes de restrições físicas:

$$V_r = V_s \cap V_a \cap V_w$$

Cada um desses subconjuntos matemáticos isola uma limitação física específica da planta robótica:

1. **Espaço de Velocidades Possíveis (V_s):** Representa o limite absoluto de saturação dos atuadores (motores DC das rodas), mapeando o intervalo operacional máximo e mínimo homologado pelo fabricante do robô:

$$V_s = \{(v, \omega) | v \in [v_{\min}, v_{\max}] \wedge \omega \in [\omega_{\min}, \omega_{\max}]\}$$

2. **Espaço de Velocidades Admissíveis (V_a):** Corresponde ao conjunto de velocidades seguras que garantem que o robô seja fisicamente capaz de desacelerar e efetuar uma parada completa antes de colidir com o obstáculo mais próximo detectado em sua projeção geométrica. Sendo $dist(v, \omega)$ a

menor distância escalar até um obstáculo ao longo da trajetória curva gerada pelo par (v, ω) , e $v_{desacel}$ e $\omega_{desacel}$ as taxas máximas de frenagem, define-se:

$$V_a = \{(v, \omega) | v \leq \sqrt{2 \cdot \text{dist}(v, \omega) \cdot v_{desacel}} \wedge \omega \leq \sqrt{2 \cdot \text{dist}(v, \omega) \cdot \omega_{desacel}}\}$$

3. **Janela Dinâmica (V_w):** É o conjunto que restringe a busca às velocidades que podem ser efetivamente atingidas dentro do próximo intervalo de tempo discreto Δt , dadas as capacidades instantâneas de aceleração e torque cinemático do robô. Considerando as velocidades reais atuais (v_a, ω_a) e as acelerações máximas v_{max} e ω_{max} , a janela dinâmica delimita-se por:

$$V_w = \{(v, \omega) | v \in [v_a - v_{max} \Delta t, v_a + v_{max} \Delta t] \wedge \omega \in [\omega_a - \omega_{max} \Delta t, \omega_a + \omega_{max} \Delta t]\}$$

O método consiste em gerar um "Espaço de Velocidades" admissíveis, combinações de velocidade linear (v) e angular (ω) que o robô fisicamente consegue alcançar no próximo instante t , respeitando seus limites de aceleração. Para cada par (v, ω) , o algoritmo projeta a trajetória e atribui uma nota utilizando a seguinte Função Objetivo:

$$G(v, \omega) = \alpha \cdot \text{Heading}(v, \omega) + \beta \cdot \text{Clearance}(v, \omega) + \gamma \cdot \text{Velocity}(v, \omega)$$

- **Heading:** Avalia o alinhamento angular do robô em relação à coordenada do objetivo final.
- **Clearance:** Avalia a distância do robô até o obstáculo mais próximo nessa trajetória simulada.
- **Velocity:** Premia trajetórias mais rápidas para garantir que o robô não fique parado caso o caminho esteja livre.

A trajetória que obtiver a maior pontuação G é escolhida e convertida em velocidades para as rodas direita e esquerda através da cinemática inversa do robô diferencial.

3. Metodologia e Implementação

3.1. Sensores e Planta Robótica

A base robótica utilizada neste projeto foi o modelo tridimensional do iRobot Create 2. A percepção do ambiente ao redor do veículo foi estruturada utilizando 5 sensores de proximidade (simulando sensores ultrassônicos do tipo HC-SR04), configurados com um alcance máximo de detecção de 0,4 metros. Estes sensores foram estrategicamente dispostos na região frontal do chassi e nomeados de acordo com sua posição: Esquerda, Diagonal Esquerda, Meio, Diagonal Direita e Direita.

Para a aplicação das equações de cinemática inversa diferencial, foram adotados os parâmetros físicos reais do fabricante para a planta robótica: o raio das rodas motrizes foi definido como $R = 0,036$ m e a distância de separação entre os eixos das rodas foi definida como $L = 0,235$ m.

3.2. Infraestrutura de Comunicação e Sincronismo Absoluto

As arquiteturas de controle propostas nas literaturas fornecidas como referência utilizam wrappers legados de comunicação assíncrona (como a antiga `remoteApi` baseada em buffers circulantes). Em testes preliminares, tal modelo demonstrou um atraso de transporte (*delay*) prejudicial: os dados coletados pelos sensores refletiam o estado passado do ambiente, resultando em colisões e travamento do robô devido ao atraso no processamento.

Para solucionar esse problema técnico, este projeto migrou toda a infraestrutura de comunicação para a **ZeroMQ Remote API** (`coppelasim_zmqremoteapi_client`). Adicionalmente, o ecossistema de simulação foi configurado para operar em **Modo Síncrono Estrito** por meio do comando:

Python

```
client.setStepping(True)
```

Sob esta topologia, o motor de física do CoppeliaSim interrompe sua linha de tempo e congela o ambiente a cada passo discreto de simulação. O simulador aguarda obrigatoriamente que o script Python colete as posições, processe os sensores, execute a varredura da janela dinâmica e despache os comandos de velocidade através do método `client.step()`. Essa abordagem elimina o *jitter* e garante um controle determinístico absoluto em tempo real.

3.3. Solução do Problema de Alinhamento de Referencial

Durante a fase inicial de testes, constatou-se empiricamente que o robô executava uma trajetória oposta, tendendo a fugir do alvo em vez de persegui-lo. Essa observação do comportamento cinemático em malha fechada indicou uma divergência de referenciais: a "frente" física do modelo 3D (a orientação adotada para a fixação dos sensores) operava com uma defasagem de 180° (π radianos) em relação ao eixo matemático padrão interpretado pelo ambiente do simulador CoppeliaSim.

Para corrigir esse desalinhamento de forma prática, sem a necessidade de reexportar as malhas do modelo 3D no software, aplicou-se uma compensação direta no código de controle. O algoritmo passou a somar π radianos à leitura nativa da orientação angular do robô, alinhando o referencial de cálculo com o referencial de movimento real:

Python

```
ori = sim.getObjectOrientation(robotBase, -1)
theta = ori[2] + math.pi
```

3.4. Otimização do Mapeamento de Obstáculos

Abordagens tradicionais de DWA exigem que distâncias puras de sensores sejam convertidas em coordenadas cartesianas globais (X, Y) para alimentar matrizes de grade de ocupação. Esse processo introduz *overhead* computacional e vulnerabilidade a ruídos de amostragem, criando barreiras inexistentes que confundem o robô.

Nesta implementação, o processamento foi simplificado: as distâncias escalares brutas retornadas pelos sensores de proximidade foram mapeadas de forma direta. Definindo um limiar geométrico de 0,4 metros, qualquer leitura abaixo desse limite atua como um fator de penalização linear na métrica de Clearance do laço de busca principal ($score = min.dist / 0.4$), reduzindo drasticamente o custo computacional por frame. Adicionalmente, para mitigar colisões causadas por pontos cegos angulares, aplicou-se um critério rigoroso de sub-limiars por alas estruturais: leituras inferiores a 0,3 metros nas alas laterais (Esquerda/Direita) ou a 0,25 metros na ala frontal central aplicam uma penalidade estática severa de -1.0 diretamente no *score* candidato. Essa restrição atua de forma direcional e condicional: a

penalidade lateral só é ativada se a trajetória candidata tentar curvar o robô na direção da parede detectada ($w > 0$ para a esquerda ou $w < 0$ para a direita), enquanto a penalidade frontal central só é aplicada em velocidades de avanço linear significativas ($v > 0.1$ m/s), forçando o algoritmo a rejeitar curvas que fizessem contato nas quinas dos obstáculos apenas quando houver intenção real de movimento naquela direção.

3.5. Comportamento de Recuperação (Fuga de Mínimo Local)

Modelos reativos baseados no DWA clássico frequentemente caem em armadilhas de mínimo local, fenômeno observado de forma crônica quando o robô tentava navegar em cantos estreitos com paredes em ângulos de 90° . Nesses cenários específicos, a alta penalidade imposta pelo algoritmo ao tentar girar (o que aproximava os sensores laterais das paredes) superava e cancelava a recompensa de atração do alvo, gerando uma perda de convergência e fazendo o robô estagnar.

Para solucionar essa limitação teórica, foi implementado um **Mecanismo Supervisor Reativo de Fuga** (*Recovery Behavior*) operando de forma externa ao loop do DWA. Uma condicional foi estruturada para interceptar a Função Objetivo clássica sempre que a menor distância detectada por qualquer um dos 5 sensores frontais decaísse abaixo da zona crítica de 0,18 metros. Nessa condição de emergência:

1. A velocidade linear frontal é instantaneamente zerada ($v = 0$ m/s).
2. O algoritmo computa o balanço aritmético do somatório de distâncias da ala esquerda contra a ala direita para identificar de forma clara o quadrante de escape livre.
3. Aplica-se uma velocidade angular pura ($\omega = \pm 0,8$ rad/s), forçando uma rotação agressiva no próprio eixo do robô até que seu campo de visão frontal seja totalmente desobstruído, permitindo que o DWA reassuma o planejamento global de forma segura.

4. Resultados e Discussão

A validação experimental em malha fechada do algoritmo expandido comprovou a eficácia das soluções propostas para corrigir os problemas observados. O estabelecimento do Modo Síncrono Estrito foi o principal pilar para o sucesso dos testes, proporcionando leituras limpas de telemetria.

Ao longo de múltiplas simulações com distribuições variadas de obstáculos artificiais, o iRobot Create 2 demonstrou capacidade de navegação fluida e contínua. Com a parametrização refinada dos pesos da função objetivo ($\alpha=2,0$; $\beta=1,5$; $\gamma=0,5$), o robô exibiu um comportamento de condução ideal: priorizava velocidades máximas em trechos abertos e reduzia a aceleração de translação de forma suave ao se aproximar de quinas, realizando curvas amplas e seguras. A normalização rigorosa do erro angular impediu oscilações matemáticas destrutivas causadas pela transição abrupta de descontinuidade de quadrantes entre $+180^\circ$ e -180° .

O funcionamento do Mecanismo Supervisor de Fuga foi testado com sucesso ao introduzir o veículo em armadilhas de beco sem saída. Em todos os ensaios, ao atingir o limite crítico de metros, o robô interrompeu seu avanço linear imediatamente, realizou o giro puro sobre seu próprio eixo em direção ao quadrante de maior Clearance e retornou ao planejamento reativo normal do DWA, alcançando o objetivo final com erro posicional inferior a metros.

5. Anexo: Código Fonte (Python)

Abaixo encontra-se a implementação completa utilizada na validação dos testes:

Python

```
# Algoritmo 2 - Éricles
import math
import time
from coppeliasim_zmqremoteapi_client import RemoteAPIClient

def main():
    print("Conectando ao Coppeliasim...")
    client = RemoteAPIClient()
    sim = client.require('sim')

    print("Conectado! Buscando handles...")

    motorEsquerdo = sim.getObject("/Cuboid/MOTOR_ESQUERDO")
    motorDireito = sim.getObject("/Cuboid/MOTOR_DIREITO")

    sensores = [
        sim.getObject("/Cuboid/SENSOR_ESQUERDO"),
        sim.getObject("/Cuboid/SENSOR_DIAG_ESQUERDO"),
        sim.getObject("/Cuboid/SENSOR_MEIO"),
        sim.getObject("/Cuboid/SENSOR_DIAG_DIREITO"),
        sim.getObject("/Cuboid/SENSOR_DIREITO")
    ]

    goalHandle = sim.getObject("/Target")
    robotBase = sim.getObject("/Cuboid")

    R = 0.036
    L = 0.235

    max_v = 0.1
    min_v = -0.05
    max_w = 0.5
```

```
max_accel_v = 0.5
max_accel_w = 1.5

alpha = 2.0
beta = 1.5
gamma = 0.5

v_atual = 0.0
w_atual = 0.0

tempo_acumulado = 0.0
intervalo_print = 2.0

client.setStepping(True)
sim.startSimulation()
print(f"Simulação iniciada. Atualizando terminal a cada {intervalo_print}s.")

try:
    while True:
        dt = sim.getSimulationTimeStep()

        # Acumula o tempo decorrido na simulação
        tempo_acumulado += dt

        pos = sim.getObjectPosition(robotBase, -1)
        goal_pos = sim.getObjectPosition(goalHandle, -1)

        # Verificação de Chegada
        dist_to_goal = math.hypot(goal_pos[0] - pos[0], goal_pos[1] - pos[1])
        if dist_to_goal < 0.2:
            print("\n[EVENTO] Alvo alcançado!")
            sim.setJointTargetVelocity(motorEsquerdo, 0.0)
            sim.setJointTargetVelocity(motorDireito, 0.0)
            client.step()
            break
```

```

distancias_obstaculos = []
obstaculos_detectados = 0
for i in range(5):
    res, dist, point, obj, n = sim.readProximitySensor(sensores[i])
    if res > 0 and dist < 0.4:
        distancias_obstaculos.append(dist)
        obstaculos_detectados += 1
    else:
        distancias_obstaculos.append(0.4)

v_min_dw = max(min_v, v_atual - max_accel_v * dt)
v_max_dw = min(max_v, v_atual + max_accel_v * dt)
w_min_dw = max(-max_w, w_atual - max_accel_w * dt)
w_max_dw = min(max_w, w_atual + max_accel_w * dt)

melhor_v = 0.0
melhor_w = 0.0
max_score = -9999.0

ori = sim.getObjectOrientation(robotBase, -1)
theta = ori[2] + math.pi

v_step = (v_max_dw - v_min_dw) / 5.0 if v_max_dw > v_min_dw else 0.1
w_step = (w_max_dw - w_min_dw) / 10.0 if w_max_dw > w_min_dw else 0.1

v = v_min_dw
while v <= v_max_dw:
    w = w_min_dw
    while w <= w_max_dw:
        theta_futuro = theta + w * 1.0
        x_futuro = pos[0] + v * math.cos(theta_futuro) * 1.0
        y_futuro = pos[1] + v * math.sin(theta_futuro) * 1.0

        angulo_alvo = math.atan2(goal_pos[1] - y_futuro, goal_pos[0] - x_futuro)
        erro_heading = abs(math.atan2(math.sin(angulo_alvo - theta_futuro),
math.cos(angulo_alvo - theta_futuro)))
        score_heading = 1.0 - (erro_heading / math.pi)

```

```

min_dist = min(distancias_obstaculos)
score_clearance = 1.0

if min_dist < 0.4:
    score_clearance = min_dist / 0.4
    if (distancias_obstaculos[0] < 0.3 or distancias_obstaculos[1] < 0.3)
and w > 0:
        score_clearance -= 1.0
        if (distancias_obstaculos[3] < 0.3 or distancias_obstaculos[4] < 0.3)
and w < 0:
            score_clearance -= 1.0
            if distancias_obstaculos[2] < 0.25 and v > 0.1:
                score_clearance -= 1.0

score_vel = v / max_v if max_v > 0 else 0
score = (alpha * score_heading) + (beta * score_clearance) + (gamma *
score_vel)

if score > max_score:
    max_score = score
    melhor_v = v
    melhor_w = w
    w += w_step
    v += v_step

dist_minima_geral = min(distancias_obstaculos)
if dist_minima_geral < 0.18:
    melhor_v = 0.0
    espaco_esquerda = distancias_obstaculos[0] + distancias_obstaculos[1]
    espaco_direita = distancias_obstaculos[3] + distancias_obstaculos[4]
    if espaco_esquerda > espaco_direita:
        melhor_w = 0.8
    else:
        melhor_w = -0.8

v_atual = melhor_v

```

```

w_atual = melhor_w

vel_esq = (v_atual - (w_atual * L / 2)) / R
vel_dir = (v_atual + (w_atual * L / 2)) / R

sim.setJointTargetVelocity(motorEsquerdo, vel_esq)
sim.setJointTargetVelocity(motorDireito, vel_dir)

if tempo_acumulado >= intervalo_print:
    print(f"[STATUS] Alvo: {dist_to_goal:.2f}m | Obstáculos:
{obstaculos_detectados}/5 | Vel: {v_atual:.2f} m/s")
    tempo_acumulado = 0.0

client.step()

except KeyboardInterrupt:
    print("\nSimulação interrompida pelo VS Code.")
finally:
    sim.setJointTargetVelocity(motorEsquerdo, 0.0)
    sim.setJointTargetVelocity(motorDireito, 0.0)
    sim.stopSimulation()
    print("Conexão encerrada.")

if __name__ == '__main__':
    main()

```

6. REFERÊNCIA

FOX, Dieter; BURGARD, Wolfram; THRUN, Sebastian. "The dynamic window approach to collision avoidance". *IEEE Robotics & Automation Magazine*, v. 4, n. 1, p. 23-33, 1997.

SANTANA, Ismael Cruz de. "Modelagem e Simulação do Robô iCreate 2 no CoppeliaSim". Trabalho de Conclusão de Curso, Engenharia Mecânica, UNIVASF, Juazeiro-BA, 2025.

RODRIGUES, Danilo Ribeiro et al. "Trabalho 1 - Janela Dinâmica". Disciplina de Tópicos Avançados em Automação, Engenharia da Computação, UNIVASF, Juazeiro-BA, 2021.

SAKAI, Atsushi et al. "PythonRobotics: a Python code collection of robotics algorithms". Disponível em: <https://arxiv.org/abs/1808.10703>.

COPPELIA ROBOTICS. "CoppeliaSim User Manual".

FERNANDEZ, B.; CERRADA, J. A.; HERREIRA, P. J. "A Simplified Optimal Path Following Controller for an Agricultural Skid-Steering Robot", 2019.

IROBOT CORPORATION. "iRobot Create 2 Anatomy & Anatomy Guide".